Module 12 - chater 14

Automate Provisioning EC2 with Terraform - Part 3

In this lecture the files used are committed to the following project branch towards the end of the video:

https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/deploy-to-ec2-default-components

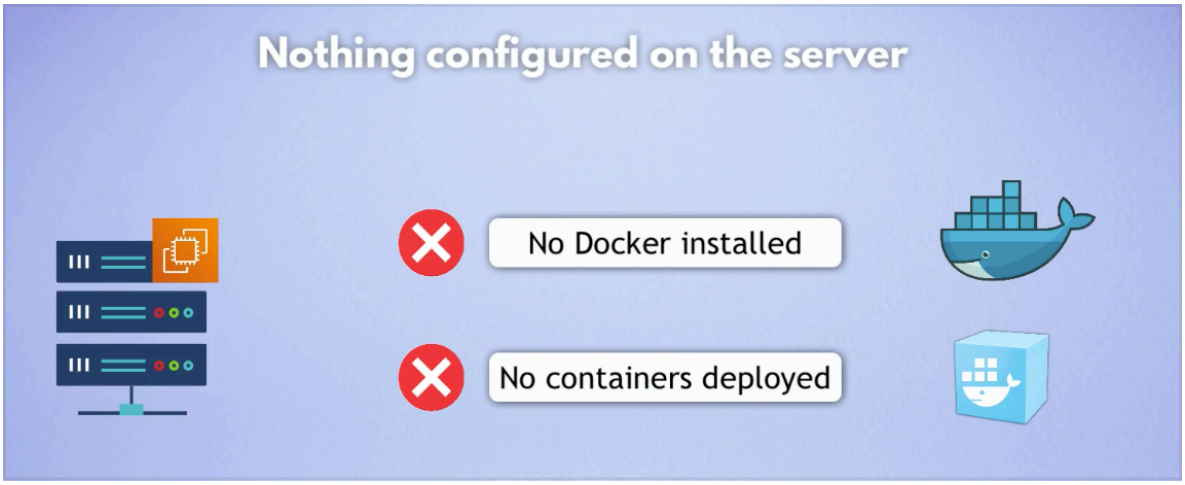
5. Deploy docker container for nginx

Run entry point script to start Docker container

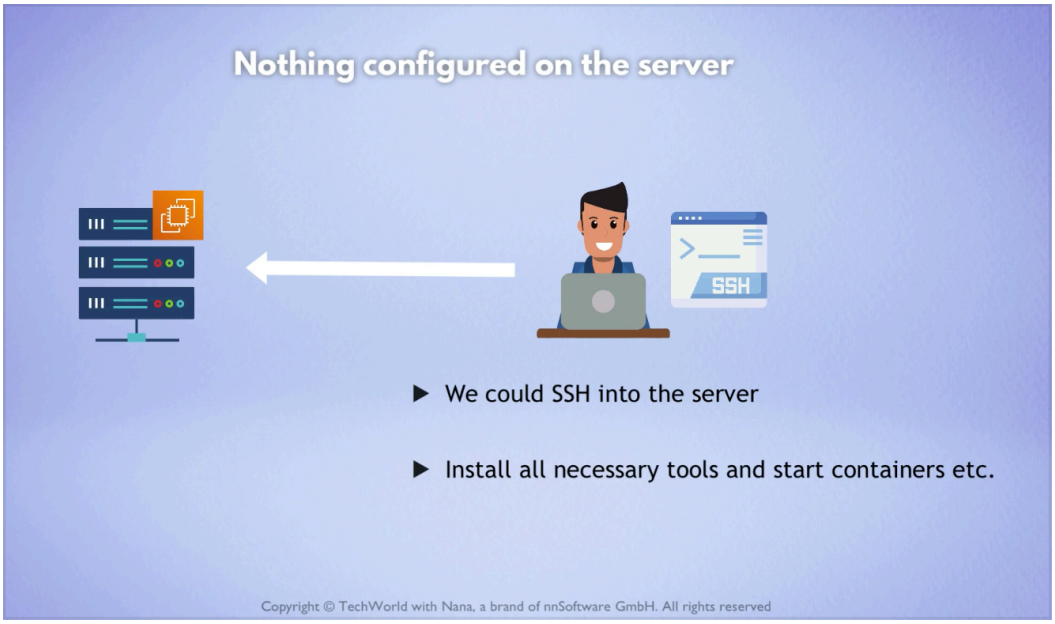
We have and EC2 and networking configured, next we need to run something on the server.

The server is empty at this point, it doesn't have:

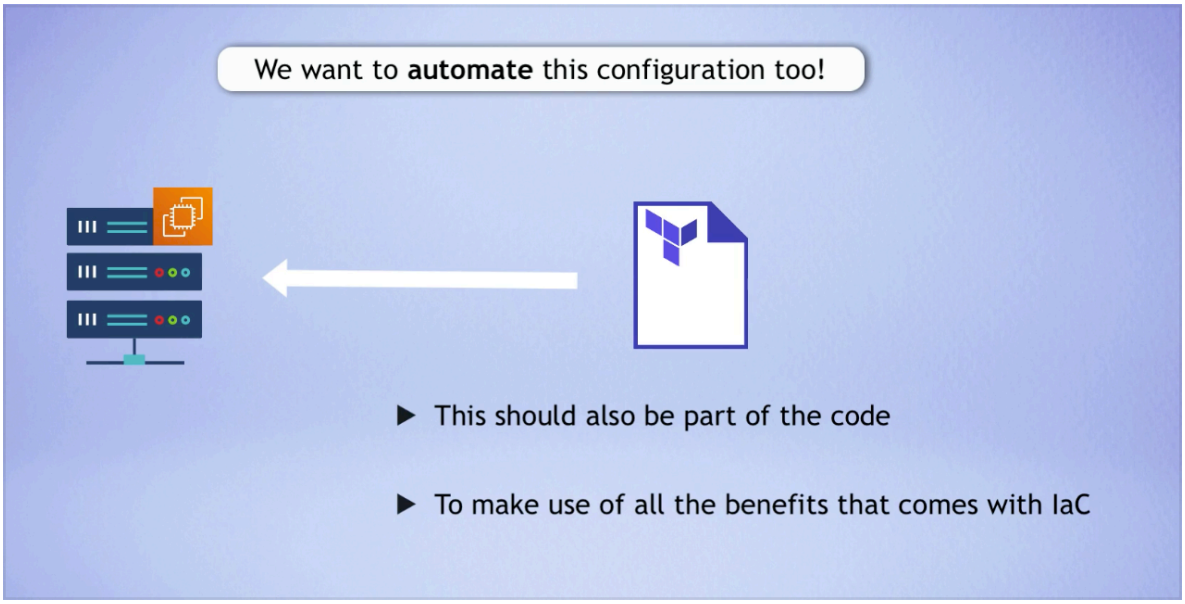
- Docker installed
- NO containers deployed



We can SSH into the server and make the necessary installations

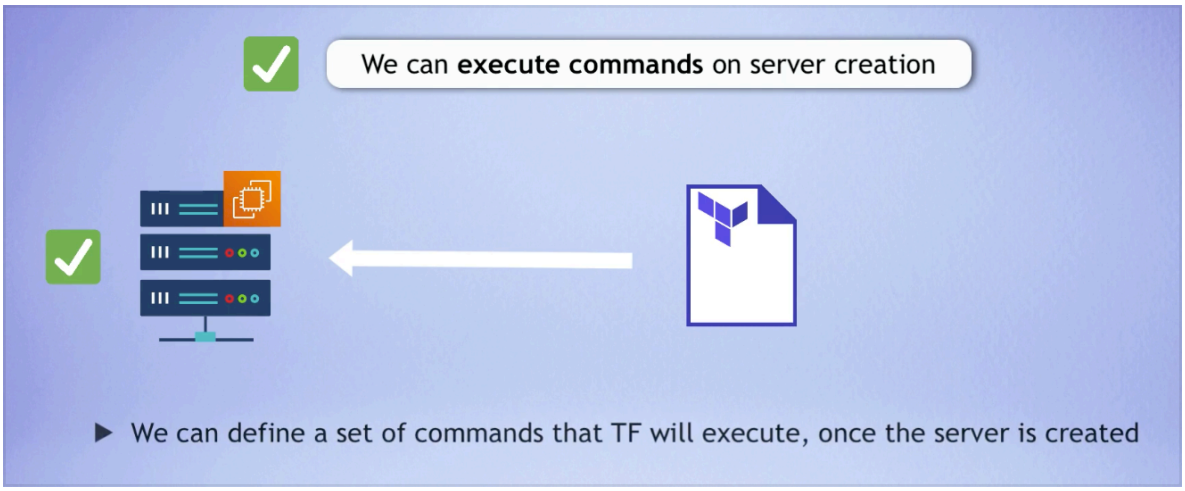


But we don't want to do manual work -> we want to automate as much as possible



And we want that in our TF code so we can benefit from IaC

With TF we can execute commands on server creation



For the above we use an attribute called ‘user_data’

```
128  resource "aws_instance" "myapp-server" {
129    ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
130    instance_type = var.instance_type
131
132    subnet_id = aws_subnet.myapp-subnet-1.id
133    vpc_security_group_ids = [aws_default_security_group.default-sg.id]
134    availability_zone = var.avail_zone
135
136    associate_public_ip_address = true
137    key_name = aws_key_pair.ssh-key.key_name
138
139    user_data =    You, Yesterday • Uncommitted changes
140
141    tags = {
142      Name = "${var.env_prefix}-server"
143    }
144  }
```

‘user_data’ is an entry point script -> gets executed whenever the server is instantiated.

We can use multi line string syntax -> in TF is called heredoc

— tangent —

Q:
TF multi line string syntax

A:
In Terraform it's called a heredoc and uses <<EOT and EOT as delimiters:

```
user_data = <<EOT
#!/bin/bash
apt-get update
apt-get install -y nginx
EOT
```

- <<EOT — opens the multiline string (EOT can be any word, it's just a convention)
- Everything between is treated as a string including newlines
- EOT alone on a line — closes the string

A common use case in Terraform is the user_data attribute on EC2 instances, which lets you run a shell script when the instance first starts up. Is that what you're working on?

Q:
› my teacher is using EOF

A:
● Yes, EOF and EOT are interchangeable — the word itself doesn't matter, it just needs to match at the opening and closing:

```
user_data = <<EOF
#!/bin/bash
apt-get update
EOF
```

- EOT stands for End Of Text
- EOF stands for End Of File

Both are just conventions — you could even use any word like MYSTRING as long as it matches:

```
user_data = <<MYSTRING
#!/bin/bash
MYSTRING
```

EOF is probably more common so it makes sense your teacher uses it.

— end of tangent ——

So we start and end the block with EOF

Then we add the cade like we do with any other bash script. And we start adding the shebang (#!)

So when you write a user_data script for an EC2 instance starting
-> **#!/bin/bash** -> it works because EC2 instances run Amazon Linux (or whatever Linux AMI you chose) — **bash** is always available there.

And we also add
-> **sudo dnf update -y**

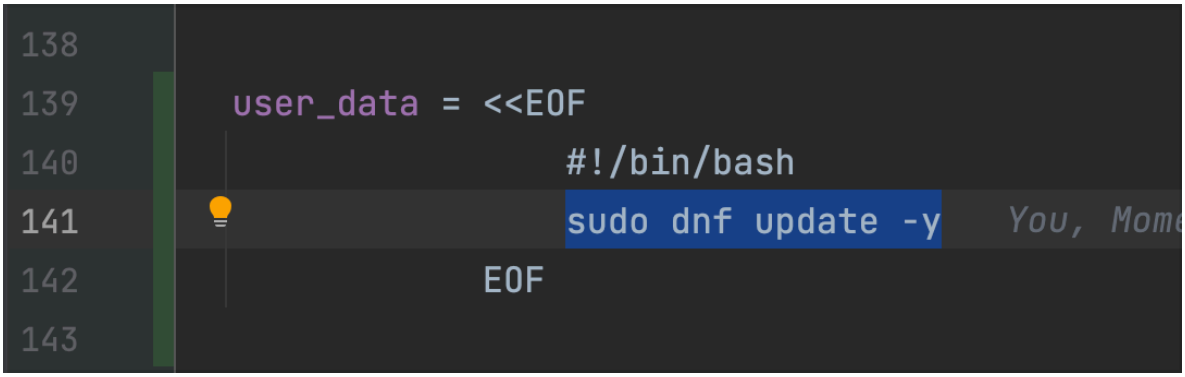
sudo -> Super User Do, runs the command with admin/root privileges. Needed because installing/updating packages requires admin rights.

dnf -> the package manager for Amazon Linux 2023 (and Red Hat based Linux)

update -> updates all installed packages to their latest versions

-y -> automatically answers yes to any confirmation prompts, so the script runs without waiting for user input. This is important in user_data scripts because there's no human there to type "y"

So the full reading is: "Run as admin, update all packages, and don't ask me to confirm anything."



Note: using def as the EC2 instance is using a fedora linux distro and this is the package manager for fedora (see questions at the end of notes)

Now we will install docker
-> && sudo dnf install -y docker

And then we need to start docker
-> sudo systemctl start docker

Now we want to be able to run docker commands without 'sudo' so we need to add the ec2-user to the docker group
-> sudo usermod -aG docker ec2-user

sudo — run as admin

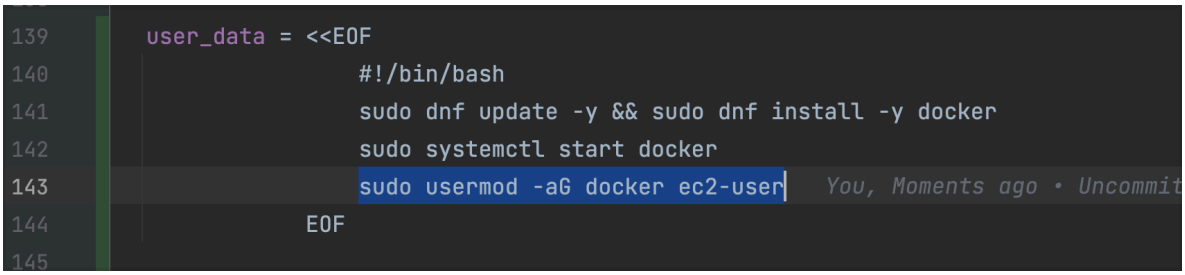
usermod — modify a user account

-aG — two flags combined:
- -a — append, add to the group without removing from other groups
- -G — specifies the group to add the user to

docker — the group being added to. On Linux, being in the docker group allows a user to run Docker commands without sudo

ec2-user — the user being modified

So the full reading is: "**Add ec2-user to the docker group without removing them from any other groups they belong to**."



Now we can run the nginx docker container
-> docker run -p 8080:80 nginx

-p 8080:80 — port mapping, two ports separated by :
- 8080 — the port on your EC2 host (outside the container)
- 80 — the port inside the container where nginx listens
- So traffic hitting your EC2 on port 8080 gets forwarded to port 80 inside the container

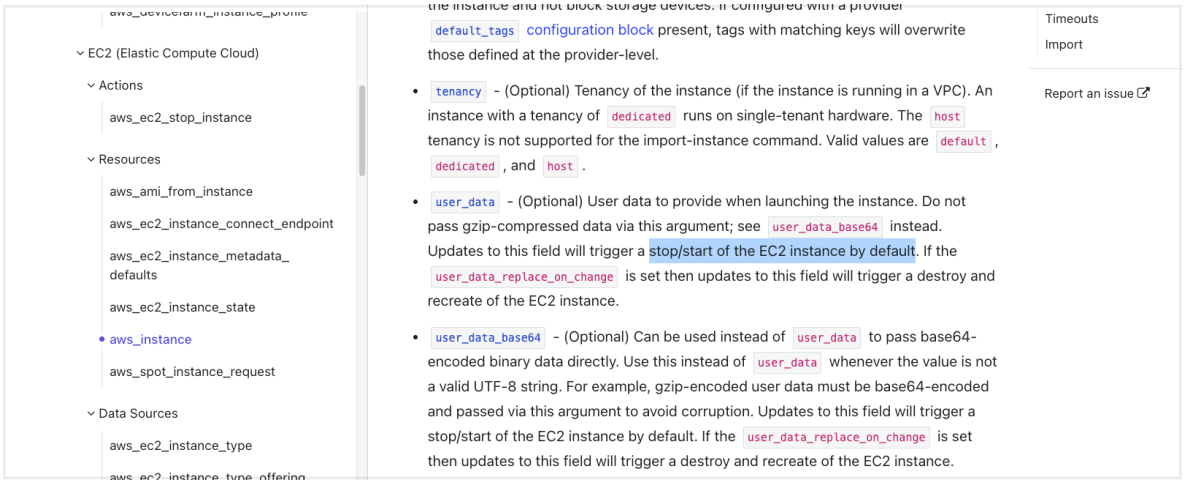
"Run an nginx container and map port 8080 on the host to port 80 inside the container."

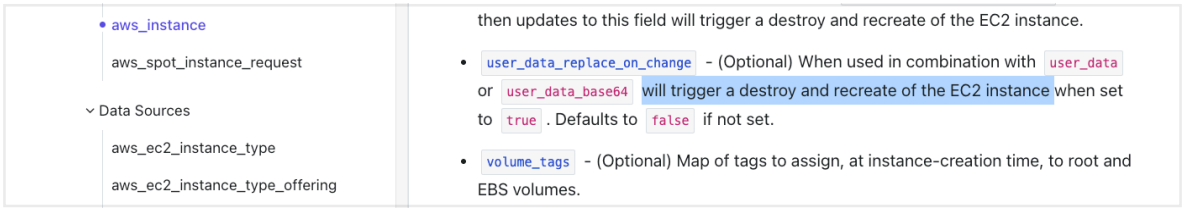
```
139   user_data = <<EOF
140               #!/bin/bash
141               sudo dnf update -y && sudo dnf install -y docker
142               sudo systemctl start docker
143               sudo usermod -aG docker ec2-user
144               docker run -p 8080:80 nginx    You, Moments ago • Uncomm
145           EOF
```

We want now to add another attribute “**user_data_replace_on_change**”.

By default when we update user_data field the EC2 instance is stopped/started -> we want too hange this, we want to destroy and re-create the whole EC2 when we update user_data field and for that we need “user_data_replace_on_change”

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance





So our script is going to run each time in a clean new instance -> consistent STATE

```
resource "aws_instance" "myapp-server" {
  ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
  instance_type = var.instance_type

  subnet_id = aws_subnet.myapp-subnet-1.id
  vpc_security_group_ids = [aws_default_security_group.default-sg.id]
  availability_zone = var.avail_zone

  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name

  user_data = <<EOF
                #!/bin/bash
                sudo dnf update -y && sudo dnf install -y docker
                sudo systemctl start docker
                sudo usermod -aG docker ec2-user
                docker run -p 8080:80 nginx
            EOF
  user_data_replace_on_change = true    You, 3 minutes ago • Uncommitted changes

  tags = {
    Name = "${var.env_prefix}-server"
  }
}
```

-> terraform plan

```
  ~ secondary_private_ips               = [] -> (known after apply)
  ~ security_groups                     = [] -> (known after apply)
  + spot_instance_request_id            = (known after apply)
    tags                                = {
        "Name" = "dev-server"
    }
  ~ tenancy                             = "default" -> (known after apply)
  + user_data                           = <<-EOT # forces replacement
        #!/bin/bash
                        sudo dnf update -y && sudo dnf install -y docker
                        sudo systemctl start docker
                        sudo usermod -aG docker ec2-user
                        docker run -p 8080:80 nginx
    EOT
  + user_data_base64                    = (known after apply)
  ~ user_data_replace_on_change         = false -> true
```

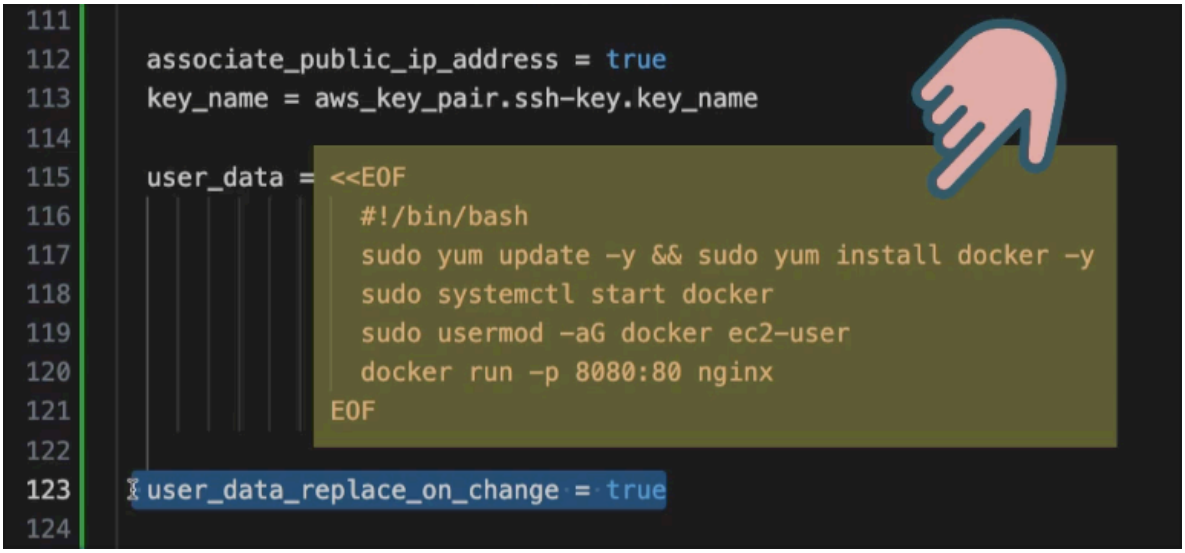
Adding user_data forces replacement (destroy -> add) of the EC2 instance

-> terraform apply

```
Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ ec2_public_ip = "18.133.235.82" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-09125d10b596caa23]
aws_instance.myapp-server: Still destroying... [id=i-09125d10b596caa23, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-09125d10b596caa23, 00m20s elapsed]
aws_instance.myapp-server: Destruction complete after 30s
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Still creating... [00m20s elapsed]
aws_instance.myapp-server: Creation complete after 21s [id=i-087be760218e1f6b2]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

ec2_public_ip = "18.135.13.63"
➜  terraform git:(Module_12/Terraform_chapter-14) ✗
```

user_data is run only once -> If I make a change, an update to the EC2 that is not the user_data like the tags, then user_data won't be executed

But as said before if I make changes to user_data then the EC2 will be recreated and user_data will be executed.



And because our TF has an output the gives us the public IP address of the EC2 instance

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

ec2_public_ip = "18.135.13.63"
➜  terraform git:(Module_12/Terraform_chapter-14) ✗
```

We don't need to go to AWS UI and we can just take that value and SSH into our EC2 instance to check the nginx container is up and running:

-> ssh ec2-user@18.135.13.63
-> docker ps
-> sudo docker ps

```
[ec2-user@ip-10-0-10-63 ~]$ docker ps
-bash: docker: command not found
[ec2-user@ip-10-0-10-63 ~]$ sudo docker ps
sudo: docker: command not found
[ec2-user@ip-10-0-10-63 ~]$
```

Command not found

Problem

Docker didn't get installed. This is a common issue with user_data scripts — they run at launch and if something goes wrong there's no feedback.

Check if the user_data script ran and what happened:
-> sudo cat /var/log/cloud-init-output.log
Or
-> sudo cat /var/log/cloud-init.log | grep -i error

This log shows everything that happened when your EC2 launched, including any errors from your user_data script.

This is the error:
2026-06-20 10:19:08,824 - subp.py[DEBUG]: Exec format error. Missing #! in script?
Reason: [Errno 8] Exec format error: b'/var/lib/cloud/instance/scripts/part-001'
raise RuntimeError(
RuntimeError: Runparts: 1 failures (part-001) in 1 attempted commands

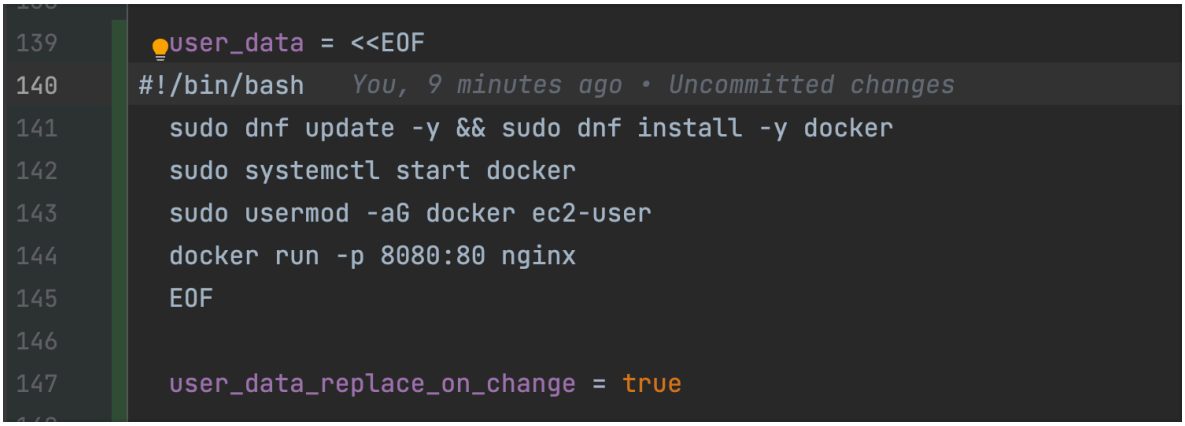
AI:
it's the indentation issue. The #!/bin/bash line has spaces before it so the OS doesn't recognize it as a valid shebang.

Fix by removing all leading spaces from the script:

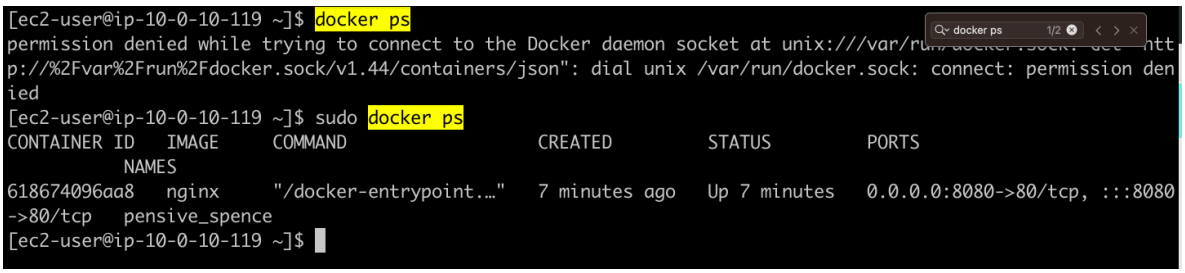
```
user_data = <<EOF
#!/bin/bash
sudo dnf update -y && sudo dnf install -y docker
sudo systemctl start docker
sudo usermod -aG docker ec2-user
sudo docker run -p 8080:80 nginx
EOF
```

The #!/bin/bash must be the very first line with no spaces or indentation — that's a hard requirement for shell scripts.

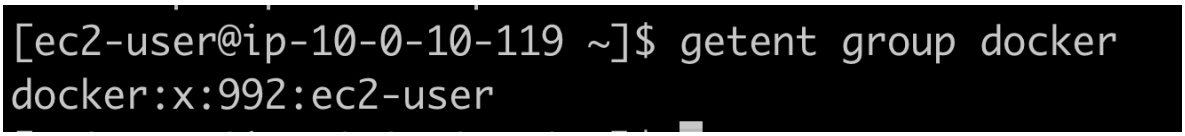
So I removed the indentation for the shebang line



And now docker is install, although I need to use 'sudo'



Let's investigate, does our user ec2-user belongs to the group 'docker'?
-> getent group docker



-> yes it does

Then what happened, why I need to use sudo?

AI:

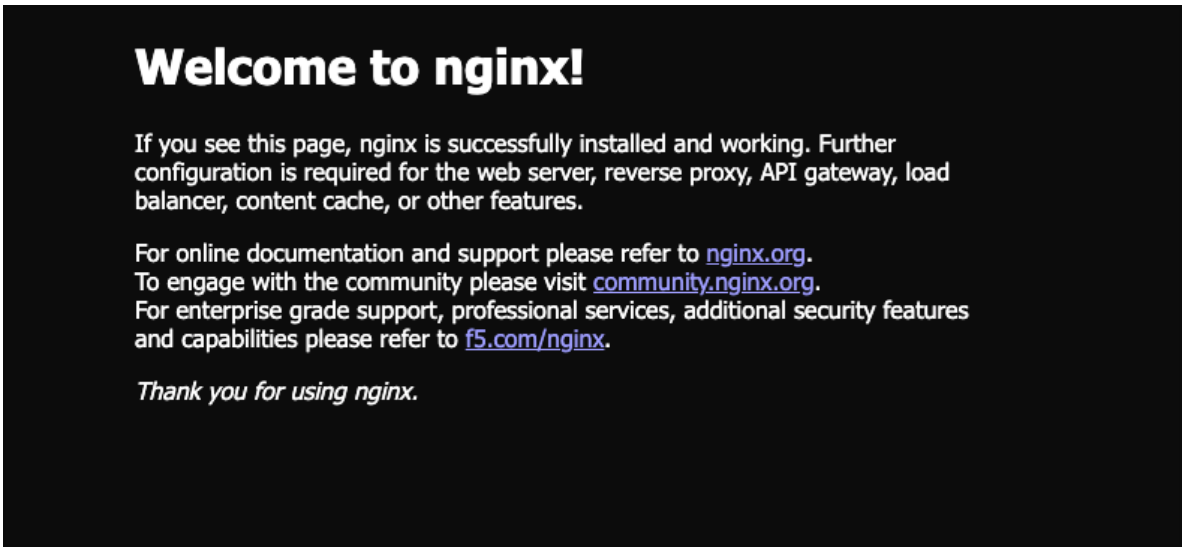
The usermod added ec2-user to the docker group, but your current SSH session was created before that change happened, so it hasn't picked it up yet.

Simply log out and log back in.

```
[ec2-user@ip-10-0-10-119 ~]$ getent group docker
docker:x:992:ec2-user
[ec2-user@ip-10-0-10-119 ~]$ exit
logout
Connection to 16.60.157.4 closed.
➜  ~ ssh ec2-user@16.60.157.4
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sat Jun 20 10:23:41 2026 from 140.228.47.252
[ec2-user@ip-10-0-10-119 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS
           NAMES
618674096aa8   nginx     "/docker-entrypoint.…"   13 minutes ago   Up 13 minutes   0.0.0.0:8080->80/tcp, :::80
80->80/tcp   pensive_spence
[ec2-user@ip-10-0-10-119 ~]$
```

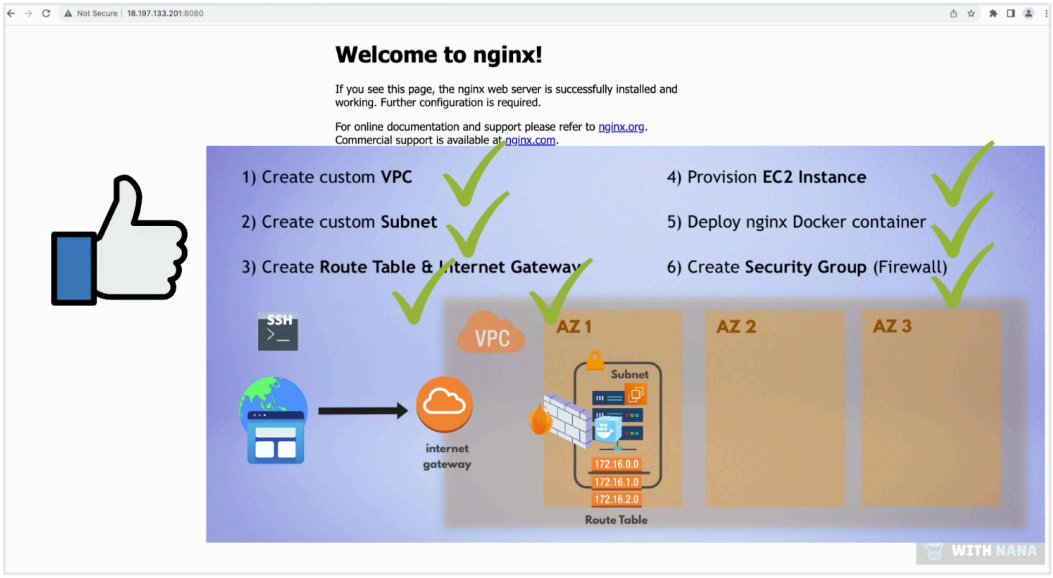
And it works! 🥳

And also we can check if we access our app via browser
-> http://16.60.157.4:8080/



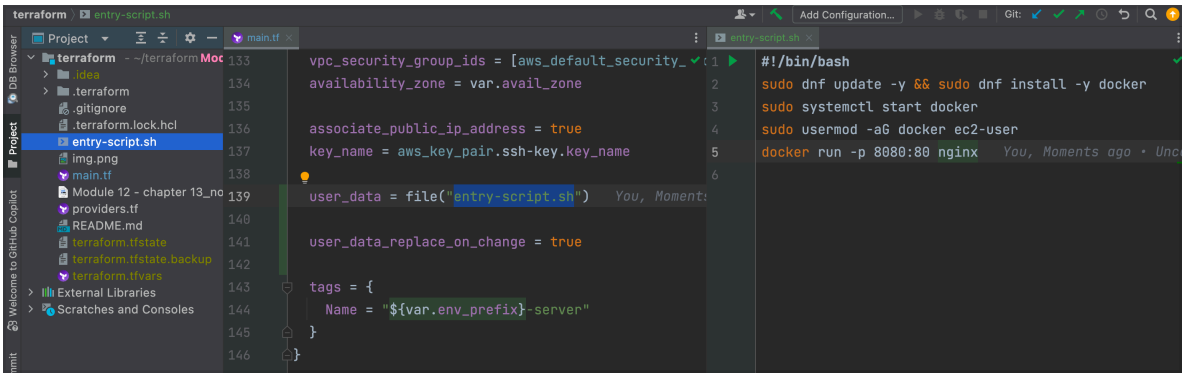
And it works! 🥳

Everything is working



Extract to shell script

We could have a longer entrypoint script with more instructions and installations that we want to execute when the server starts -> we can reference the script entry point file here instead of having the code in the TF file directly.



So I create an .sh file with our code and then I reference it in our tf file (see line 139 👆)

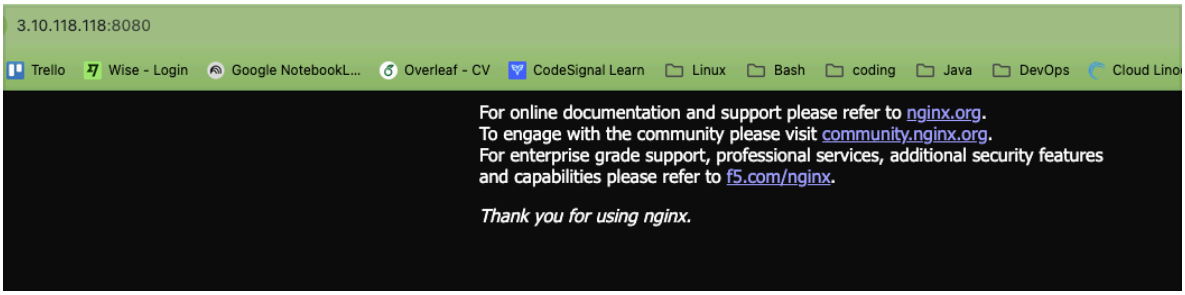
-> terraform plan

```
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated w
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # aws_instance.myapp-server must be replaced
-/+ resource "aws_instance" "myapp-server" {
      ~ arn                                  = "arn:aws:ec2:eu-west-2:706976333940:instance/i-03aff4ec7bf814522"
      ~ disable_api_stop                     = false -> (known after apply)
      ~ disable_api_termination              = false -> (known after apply)
```

-> terraform apply

```
aws_instance.myapp-server: Creation complete after 22s [id=i-08d0694104543e8f5d5]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

ec2_public_ip = "3.10.118.118"
➜  terraform git:(Module_12/Terraform_chapter-14) ✗
```

And I access nginx vis the browser (it took sometime for the container to be up and running, so I had to wait) it works 🥳



Commit to own feature branch

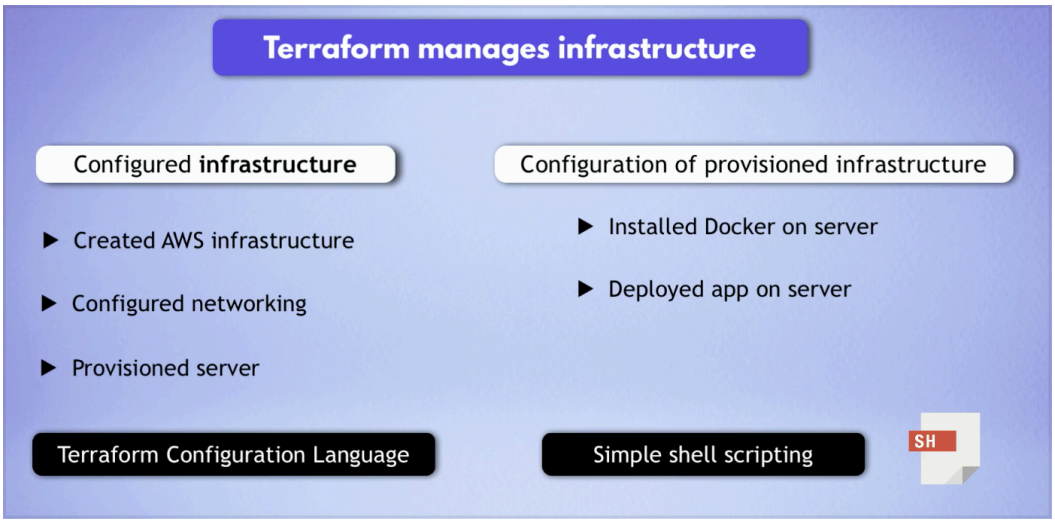
```
nana@macbook /Users/nana/terraform [master]
% git checkout -b feature/deploy-to-ec2-default-components
Switched to a new branch 'feature/deploy-to-ec2-default-components'
nana@macbook /Users/nana/terraform [feature/deploy-to-ec2-default-components]
% git add .
nana@macbook /Users/nana/terraform [feature/deploy-to-ec2-default-components]
% git commit -m "add ec2 deployment config"
[feature/deploy-to-ec2-default-components 0d61ade] add ec2 deployment config
 2 files changed, 112 insertions(+), 12 deletions(-)
 create mode 100644 entry-script.sh
nana@macbook /Users/nana/terraform [feature/deploy-to-ec2-default-components]
% git push --set-upstream origin feature/deploy-to-ec2-default-components
```

Configuring Infrastructures, not servers

TF manages infrastructure! ⭐

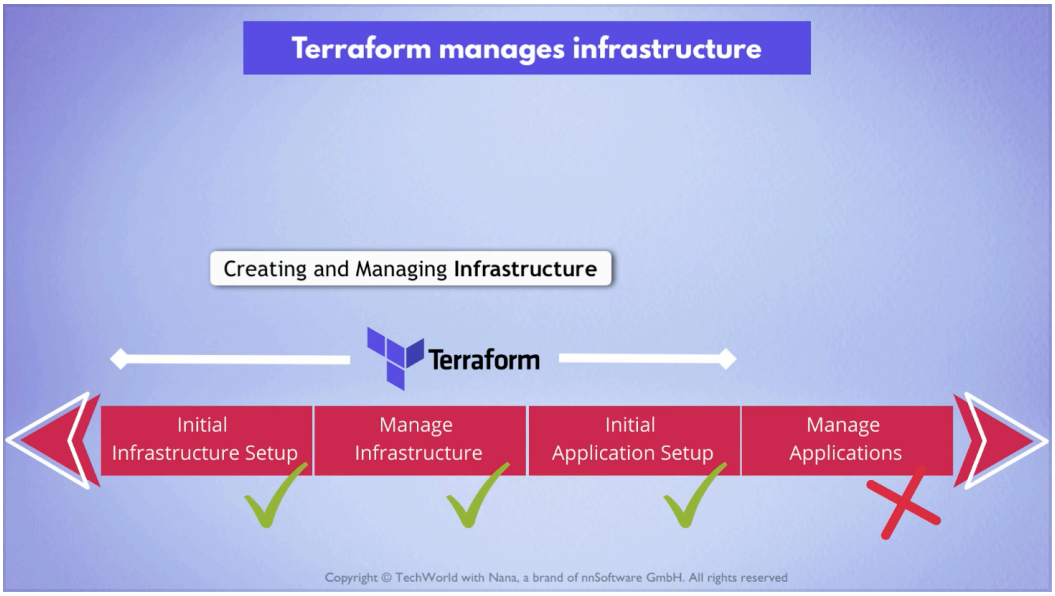
Once the infrastructure is there, once we have provisioned the server then we started using shell scripting

-> so Tf didn't help further once the infra was created.



TF gives us the attribute to use a script (user_data) but we can't debug the script with TF. If something goes wrong with our script we don't get a message, we have to ssh into the instance and figure it out like I did with the indentation issue of my shebang line (see above)

This is why it is important to know when terraform is useful and where TF usage stops



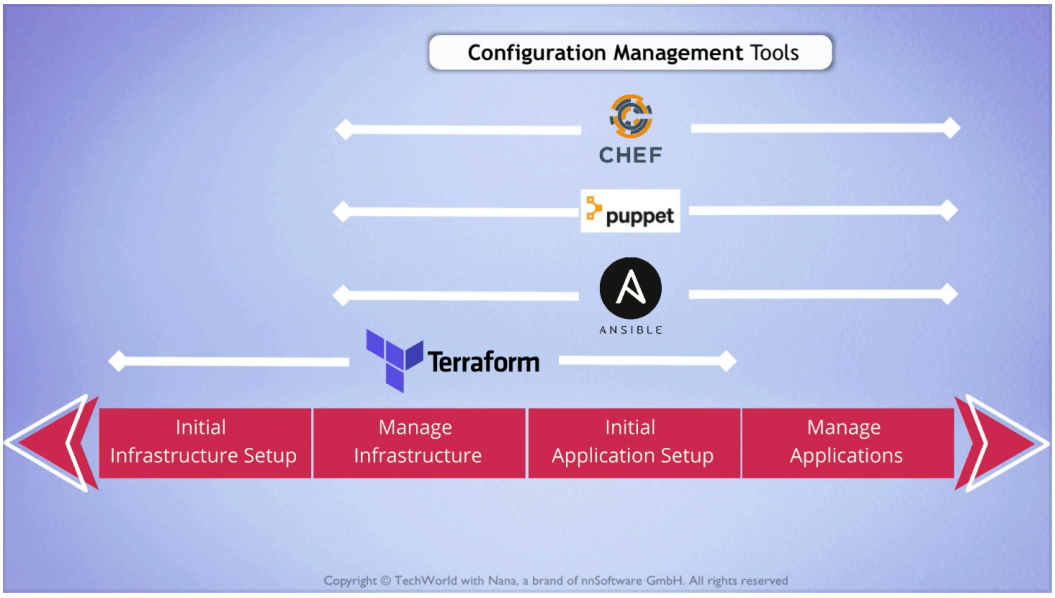
This is why TF is a tool for
Creating, configuring and managing infrastructure.

Managing the applications should be done by another tool.

At this point we used shell script, so we need to know how to write for terraform and how to write shell scripts.

Are there alternatives?

There are other IaC tools made for deploying applications and configuring exiting servers

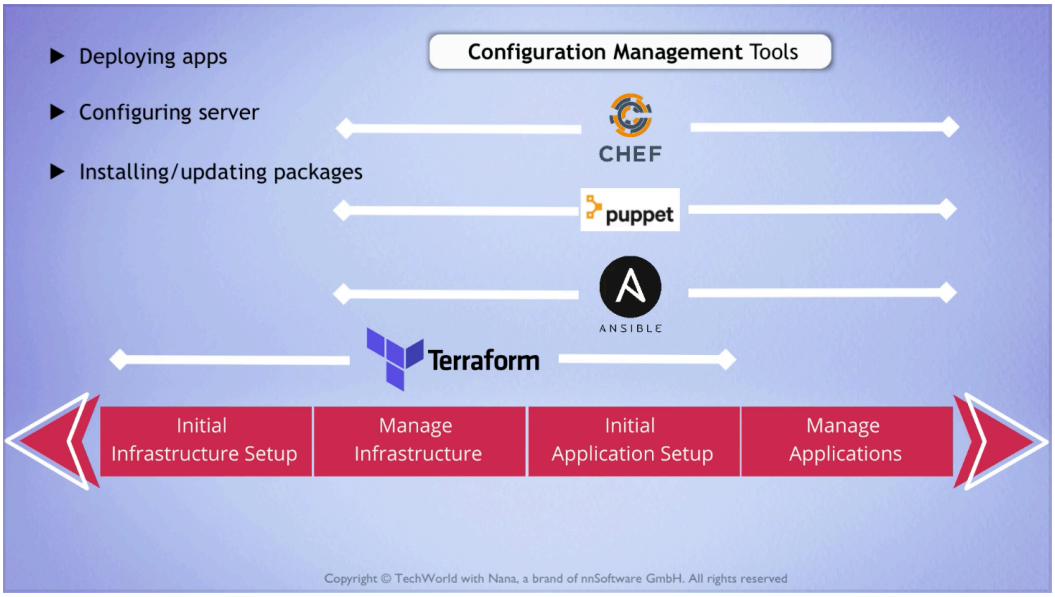


They are called configuration management tools:

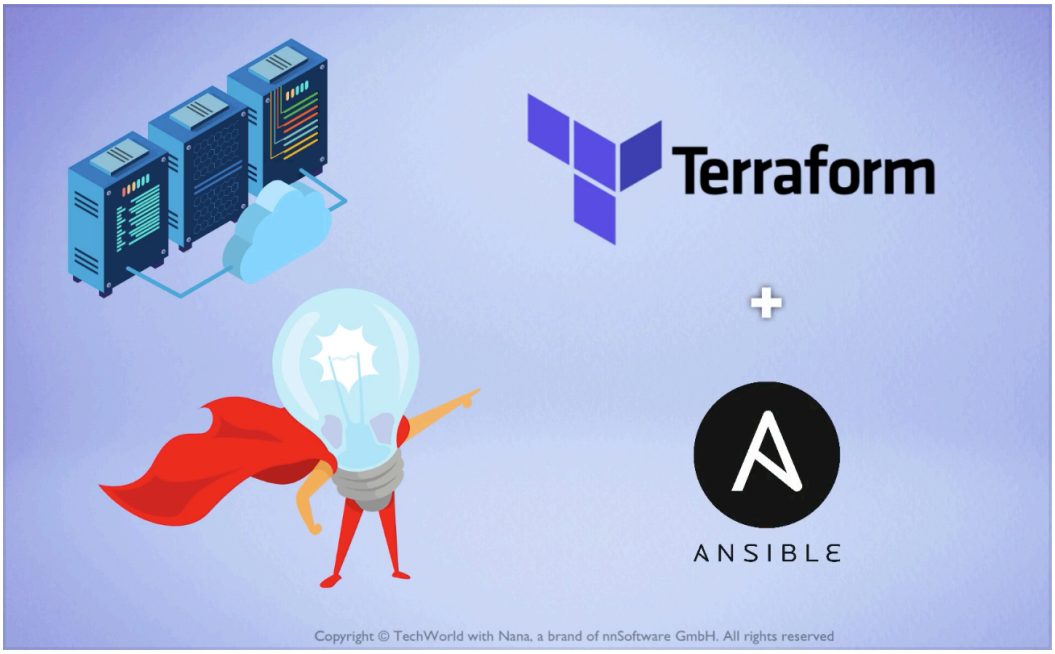
- Chef
- Puppet
- Ansible

They help us to automate:

- Deploying apps on the provisioned infra (done by TF)
- Configuring the server
- Installing different packages
- updating versions of the packages and apps



Ansible is the most popular IaC configuration management tool.



Questions

Q:
can you remind me again the shell concepts and talk about bash vs zsh

A:
● Sure! Here's a refresher:

What is a shell?
- A **program** that lets you interact with the operating system by typing

commands
  - It's the interface between you and the OS

  Bash (Bourne Again Shell):
  - Default shell on most Linux distributions
  - Been around since 1989, very widely used
  - Scripts written in bash work almost everywhere on Linux
  - That's why shell scripts start with #!/bin/bash — it tells the OS which shell to use to run the script

  Zsh (Z Shell):
  - Default shell on macOS since Catalina (2019)
  - Built on top of bash — everything that works in bash works in zsh
  - Adds extra features like:
    - Better autocomplete
    - Plugins (via Oh My Zsh)
    - Better history search
    - Themes and customization

  Key point:
  - For scripting — bash is the standard, more portable across Linux systems
  - For daily terminal use — zsh is more user friendly
  - That's why your Mac uses zsh day to day but EC2 scripts use bash

  Think of zsh as bash with extra features on top — they're very similar for basic use.

Q:
 so with zsh is not usual to write scripts?

A:
● It's not that it's unusual, it's just that bash is the standard for scripts for one main reason:

  Portability — if you write a script with #!/bin/bash it will run on virtually any Linux server, Mac, or CI/CD system without issues. If you write #!/bin/zsh it will only work on machines that have zsh
  installed, which is not guaranteed on Linux servers.

  So in practice:
  - Scripts → always use bash (#!/bin/bash) for portability
  - Interactive terminal → use zsh on your Mac for the better experience

  A good way to think about it:
  - **Zsh** is great for **you** (daily use, autocomplete, themes)

- **Bash** is great for **servers** and automation (portable, universal)

That's why your EC2 user_data scripts use bash even though your Mac uses zsh.

Q:
and yum is the package manager for linux OS?

A:
● Almost right but swapped around:

Linux package managers:
- yum — for Red Hat based Linux (Amazon Linux, CentOS, RHEL)
- apt — for Debian based Linux (Ubuntu, Debian)
- dnf — newer replacement for yum (Amazon Linux 2023 actually uses dnf)

macOS:
- Homebrew (brew) — the most popular package manager for Mac, but it doesn't come pre-installed, you have to install it yourself
- macOS has no built-in package manager like Linux does

So for your EC2 user_data scripts:
- If using Amazon Linux 2023 → use dnf (or yum still works as an alias)
- If using Ubuntu → use apt

That's why you might see scripts like:

```
#!/bin/bash
yum update -y       # Amazon Linux 2
dnf update -y       # Amazon Linux 2023
apt-get update -y   # Ubuntu
```

Q:
how do I check the package of the server using command line?

A:
● It depends what you want to check:

Check which package manager is available:

```
which dnf
which yum
which apt
```

List all installed packages:

```
dnf list installed       # Amazon Linux 2023 / Red Hat
yum list installed       # Amazon Linux 2
apt list --installed     # Ubuntu/Debian
```

Check if a specific package is installed:
dnf list installed nginx
apt list --installed | grep nginx

Check the OS/distro to know which package manager to use:
cat /etc/os-release

This last one is very useful when you SSH into an unfamiliar server and want to know what Linux distribution it's running and therefore which package manager to use.

Q:
so from cat /etc/os-release I can deduce that the linux disrto is fedora which is red hat?
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023.12.20260611"
ANSI_COLOR="0;33"
CPE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AWS"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_END="2029-06-30"

A:
● Yes, exactly! The key line is:

ID_LIKE="fedora"

This tells you Amazon Linux 2023 is based on Fedora which is a Red Hat family distro. So the family tree is:

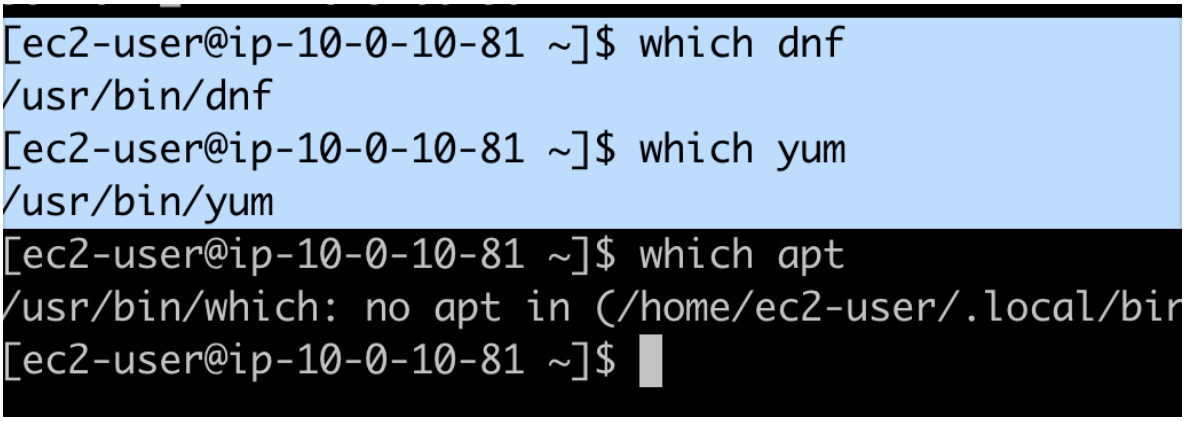
Red Hat → Fedora → Amazon Linux 2023

That's why it uses dnf (Fedora's package manager) instead of apt (Debian/Ubuntu's package manager).

A few other interesting things from that output:

- VERSION="2023" — it's Amazon Linux 2023
- SUPPORT_END="2029-06-30" — supported until 2029, which matches the 5 years of LTS you saw mentioned in the AMI description earlier
- VENDOR_NAME="AWS" — confirms it's Amazon's own Linux distribution, not pure Fedora
- PRETTY_NAME="Amazon Linux 2023.12.20260611" — the 20260611 is the build date (June 11 2026), which matches the AMI name you saw earlier

Q:
I checked yum, def and apt



It says my EC2 server has two package managers: def and yum

A:
Both are available! That's expected on Amazon Linux 2023 — yum is kept as an alias/wrapper for backwards compatibility, but underneath it actually calls dnf.